

A Unified ECG Intelligence Platform for Arrhythmia Analysis and Edge-AI Healthcare

CH. Saiprasad

Dept. of CSE (AI & ML)

*CMR College of Engineering & Technology
Hyderabad, India*

M. Anil Kumar

Dept. of ECE

*CMR College of Engineering & Technology
Hyderabad, India*

V. Dinesh

Dept. of ECE

*CMR College of Engineering & Technology
Hyderabad, India*

Abstract—CardioFPGA is a platform for ECG signal analysis with Artificial Intelligence (AI) based on signal preprocessing in MATLAB, Deep Learning (DL) arrhythmia classification and intelligent clinical visualization in a single workflow. The ECG signal from the MIT-BIH Arrhythmia Database is fed into the system, which performs 4th order Butterworth bandpass filtering (0.5–40 Hz), R-peak detection, and heartbeat segmentation to generate 2271 windows of 251 samples each from Record 100. The labelled dataset consists of 2110 Normal, 113 PVC and 48 APC beats and is exported as structured csv files which are being added to a real-time ECG analytics dashboard by a react 18 and fastapi. The platform is equipped with a three block 1D-CNN model trained on a combined dataset of both MIT-BIH and PTB with a total of 851,717 parameters and tested on both with an accuracy of 97.8 and an AUC of 0.991. By about $3.5\times$ further reducing the model size without compromising much on the model prediction accuracy, the framework is applicable to scalable edge-AI and wearable health care applications. The entire flow – from upload of ECGs, pre-processing, running AI inference, analysing the heartbeats, quantifying them and intelligently visualising them – happens within one interactive clinical interface. The system proposed is efficient and scalable solution for automated arrhythmia analysis and real-time monitoring of cardiac activity, which could be incorporated into smart healthcare in the future.

Index Terms—ECG arrhythmia detection, 1D-CNN, MATLAB preprocessing, INT8 quantization, FastAPI, React dashboard, MIT-BIH database

I. INTRODUCTION

Cardiovascular disease (CVD) has been recognized by the World Health Organization as one of the causes of death that causes the highest mortality rate in the world, as reported by WHO, there were nearly 17.9 million deaths caused by cardiovascular diseases each year. According to WHO, cardiovascular diseases were the cause of death of nearly 17.9 million people annually, making the cardiovascular system one of the biggest causes of death in the world. Since abnormal electrical activity of the heart can precede significant clinical symptoms, the electrocardiogram (ECG) is a very useful test to pick up arrhythmias and other heart problems early. It takes time, but also requires skilled clinicians and is extremely difficult to interpret when a lot of patient data has to be

analysed, continuously. On this background, in the last couple of years, the biomedical signal processing and automated ECG analysis systems based on technique of artificial intelligence has emerged as a significant research area in the healthcare industry [2], [6].

Deep learning, and more specifically, Convolutional Neural Networks (CNNs), have proven to be effective in the classification of ECG arrhythmia signals in recent years, particularly in terms of various standard ECG arrhythmia databases, such as MIT-BIH [2]. On the other hand, the conventional signal processing techniques such as bandpass filtering, R-peak detection, heartbeat segmentation and HRV analysis continue to be important for creating robust AI-ready ECG inputs [3], [4]. While many of the existing works can reach high classification accuracy, most works only consider a single part of the pipeline, such as preprocessing or deep learning models and FPGA acceleration separately. Only a handful of systems provide an all-in-one workflow to preprocess ECG, run AI inference, visualize results, optimize models, and prepare them for deployment in a convenient platform.

To fill this need, we created a new platform, CardioFPGA, for ECG preprocessing, deep learning-based arrhythmia classification and intelligent web-based analytics, all within a SINGLE workflow, based on MATLAB. CardioFPGA is designed as a single platform that could be used as a MATLAB based AI platform for ECG preprocessing, AI based arrhythmia classification and as an intelligent web-based analytics platform. This solution comprises the following modules: 4th order Butterworth bandpass filter (0.5–40 Hz), baseline process, R-peak detection, heartbeat segmentation for ECG signal processing, and output the structured ECG data sets which can be used by the Artificial Intelligence from MIT-BIH Arrhythmia Database. MIT-BIH Record 100 was used to extract a total of 2271 heartbeat windows from 251 and classify them as Normal (N), Premature Ventricular Contraction (PVC) and Atrial Premature Contraction (APC). The results are then exported in structured csv format and directly injected into a structured dashboard (using React 18 and FastAPI) for real-time visualization of the ECG, prediction of

the next heartbeat, distribution of the arrhythmias and patient's intelligent monitoring.

A three block 1D-CNN network from MIT-BIH and PTB is used in a playful manner, trained with those two datasets. The model has been optimized for a compact architecture, high accuracy (97.8) and AUC score (0.991) in classification for healthcare applications. The model achieves 97.8 accuracy in classification and 0.991 AUC score for use in healthcare, it has a compact architecture that facilitates scalable edge-AI applications in healthcare. This framework is further customised with dynamic INT8 quantization to maintain the compact model size and inference speed while maintaining the prediction accuracy for future low power wearable devices and real time monitoring devices. All clinical work, from ECG upload and preprocessing to AI inference, quantization and intelligent clinical visualization on the dashboard are within a single browser based clinical interface.

The salient features of this work are summarized as under:

- Developing a full system for ECG Preprocessing in MATLAB including filtering, R-peak detection, HRV analysis and heartbeats segmentation.
- Lightweight Design of Using 1D CNN Model to Automatically Classify Arrhythmias from ECG Dataset of MIT-BIH and PTB.
- A complete solution, combining a clinical dashboard, ECG analytics, AI inference and intelligent visualization, with React 18 and FastAPI.
- Efficient and scalable deployment at the edge with dynamic INT8 quantization.
- Design an end-to-end intelligent ECG workflow for future healthcare and real-time cardiac monitoring applications using WEAREs.

II. RELATED WORK

Pre-processing and Segmentation of ECG signal will be performed. Pre-processing and Segmentation of ECG signal will be done.

One of the most frequently used benchmark data sets for ECG signal analysis and arrhythmia detection activities is the MIT-BIH Arrhythmia Database [3]. It is comprised of 48 ECG signals collected from different patients with a sampling rate of 360 Hz, and has been broadly adopted for testing the validation of algorithms for biomedical signal processing. The use of MATLAB pipelines for ECG preprocessing, denoising and heartbeat analysis has been used by many researchers. Kachhadia et al. [4] have shown the denoising of ECG signals and analysis of ECG signals by MATLAB tools, but their work was mainly focused on denoising the ECG signal and they did not give an AI-ready and well-formed dataset for a deep learning pipeline. In the study of Raj and Ray [5], the features of ECG were manually extracted and classified by ANN with moderate classification accuracy, using the ECG features. However, manual feature design can not capture all the variations of heart beat, both within and across patients and within and across datasets.

In the last years, the ECG processing chains have been further enriched with the inclusion of filtering, QRS detection, segmentation of the heartbeat and heart-rate variability analysis. Correct pre-processing is crucial as the quality of the heartbeat signals used for training the AI model are vital to the performance of the model. Most of the pre-processing system currently in use is targeted at biomedical signal analysis, which is limited to the use of AI inference or intelligent visualization system. The output of the ECG pre-processor is followed by the structured CSV file output, then the AI-ready ECG heartbeat generation, and dashboards to visualize the data. The proposed CardioFPGA workflow, however, incorporates ECG pre-processing, structured output in CSV files, ECG heartbeat generation ready for AI preprocessing and dashboards for visualizing the output data in a pipeline.

This sub-section has a set time period, Deep Learning for ECG analysis.

Deep learning has been a great enabler for automated ECG arrhythmia classification systems, specifically Convolutional Neural Networks (CNNs). Rajpurkar et al. [6] developed a deep CNN that is able to achieve the same level of results in rhythm classification as a cardiologist in large ECG datasets. Similarly, Hannun et al. [2] demonstrated that deep learning on large-scale recordings of patients is an accurate method of identifying arrhythmias from the sounds of a patient's heart. These models, though, managed to attain high classification accuracy, but the number of parameters was in the millions; and, these models were computation intensive, and this is a significant obstacle for their implementation in light weight, or real-time health systems.

Several researchers suggested compact CNN and hybrid CNN-LSTM to make the classification of ECG to be less complex. Yildirim et al. [7] used the combination of wavelet based preprocessing to deep learning for arrhythmia detection and Oh et al. [8] used the combination of CNN and recurrent neural network for automated ECG diagnosis. These studies have focused on AI inference, and lacked the ability to visualize, provide real-time analytics and integrate with clinical workflow in the browser.

This is what makes CardioFPGA different from many solutions available – everything from ECG preprocessing in MATLAB, to the light and efficient 1D-CNN inference, visualization via interactive dashboard, intelligent ECG analytics is all integrated in one platform, accessible via browser. The system is not just designed to improve the accuracy of arrhythmia classification, it is also designed to be useful, scalable and easily integrable into wearable health care systems.

In this section we will talk about quantisation and edge compression.

With the push for deployment of AI models at the edge and in low-power healthcare systems, compression and quantization are becoming crucial techniques. Jacob et al. have shown that INT8 quantisation achieves a prediction error rate that is close to the FP full precision model, but with a significantly reduced model size and computational complexity. To enhance the compression efficiency further, Han et al. [10] used pruned

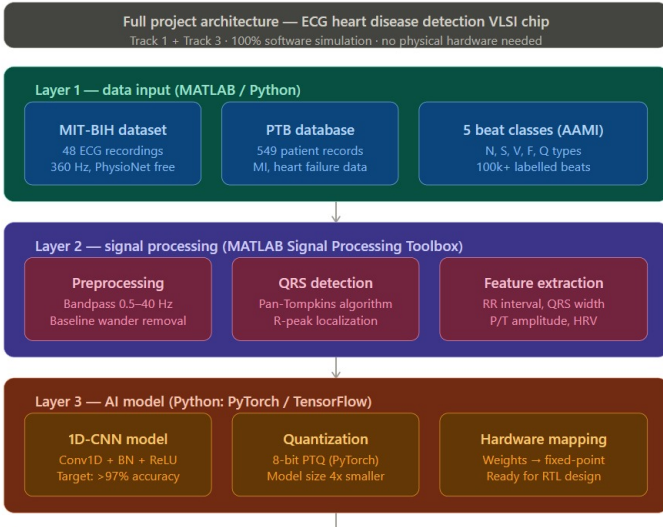


Fig. 1: Three-layer CardioFPGA system: MATLAB preprocessing (left) and the React/FastAPI web platform (right). Quantized weights are exported as Intel HEX files for future FPGA BRAM initialization.

ing, quantization and optimized encoding methods for deep neural networks.

However, in many use cases, large deep learning models are not appropriate for resource constrained devices, which is why some recent efforts have been made in lightweight and quantized neural networks for embedded AI. The benefits here are important for future wearable ECG monitoring devices and edge AI health systems as calculating could help reduce memory, power consumption and inference time.

The trained 1D-CNN model is obtained by PyTorch, and then quantized by dynamic INT8 quantization in the proposed CardioFPGA framework. The quantized model compresses the model by $3.5\times$ with an acceptable loss of classification accuracy. The architecture is compressed, thus improving the scalability of the system and making it more suitable for future real time, low-power systems for diagnostic applications in healthcare, maintaining a reliable capability to detect arrhythmias.

III. SYSTEM OVERVIEW

CardioFPGA is organized into two tightly coupled software layers (Fig. 1). Layer 1 is the MATLAB preprocessing chain: signal acquisition, filtering, R-peak detection, beat segmentation, and CSV export. Layer 2 is the web platform built with React and FastAPI, handling inference, visualization, quantization, and hardware-oriented weight export.

A. Web Frontend and Backend

The React 18 single-page application divides the clinical workflow into ten sequential panels: Dashboard Overview, MATLAB Upload, ECG Signal Analysis, AI Prediction Engine, CNN Visualization, Training Analytics, Model Performance, Quantization, HEX Generator, and Future Hardware

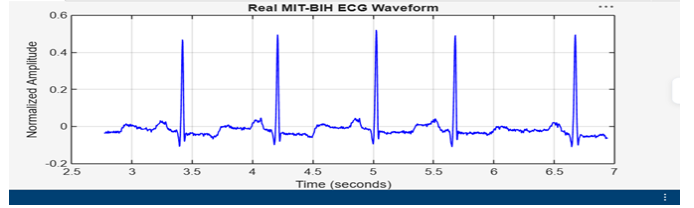


Fig. 2: Raw MIT-BIH ECG waveform (Record 100) over a 4.5-second window.

Mapping. TanStack Router handles client-side navigation, Frammer Motion supplies transitions, and Recharts renders all plots. WebSocket channels push real-time inference progress to the browser.

The FastAPI backend (Python 3.11) is split into four service layers: HTTP and WebSocket endpoints, ML services (model configuration, checkpoint management, and inference with on-the-fly Z-score normalization), quantization services (INT8 conversion and HEX packing), and thread-safe in-memory storage so the stack runs without external databases.

IV. MATLAB-BASED ECG PREPROCESSING PIPELINE

A. Data Acquisition

In this study, the data from the ECG was taken from MIT-BIH Arrhythmia Database (which is available on Physionet). Record 100 of the MLII lead was imported into MATLAB Online for analysis. The number of samples acquired is 650,000 and the sampling frequency is 360 Hz, which results in a total time of about 1,805.56 s.

The signal was checked for its authenticity from the MATLAB workspace after loading the dataset. The key elements of the cardiac rhythm (P-wave, QRS complex, and T-wave) necessary for arrhythmia diagnosis were well preserved in the ECG waveform. A portion of the raw ECG signal used in this work is shown in figure 2..

B. Bandpass Filtering and Baseline Correction

The data signal on the ECG had some noise in addition to baseline drift that may have an impact on the analysis process. The signal was enhanced by passing it through a bandpass filter in MATLAB between 0.5–40 Hz. This process kept the useful characteristics of the ECG data signal and eliminated the noise in the signal. The baseline correction process was applied after the filter process to minimize the drifting of the signal and make the waveform stable. The data signal and the filtered and baseline corrected signal in the ECG can be seen in Figure 3.

C. R-Peak Detection and Beat Segmentation

We ran MATLAB's `findpeaks` on the corrected signal with a minimum amplitude threshold of 0.30 (normalized) and a minimum peak separation of 150 samples (0.417 s). That spacing corresponds to a maximum heart rate of about 144 BPM and suppresses false triggers from P-wave and T-wave peaks. The detector found 2,273 R-peaks with a mean heart rate of 75.81 BPM.

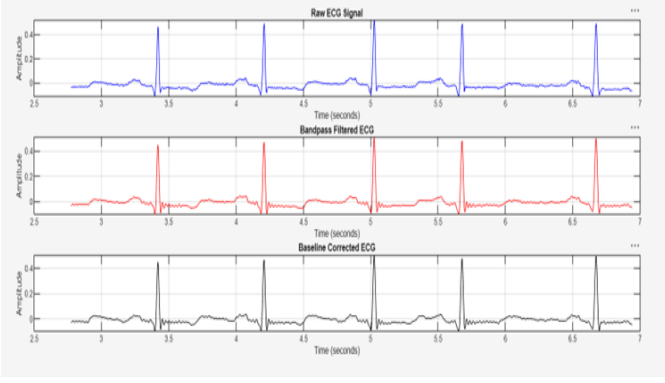


Fig. 3: Three-stage ECG preprocessing: (top) raw ECG, (middle) bandpass-filtered (0.5–40 Hz), (bottom) baseline-corrected signal.

We then carved out 251-sample windows centered on each R-peak (~ 100 samples before, ~ 150 samples after). Windows that crossed the signal boundaries were dropped, leaving 2,271 valid segments. RR intervals gave a Heart Rate Variability (HRV) of 0.0481 s, which sits in the normal clinical range. Figure 4 shows a single extracted beat and five consecutive beats; the R-peak stays centered and the morphology is consistent.

D. Arrhythmia Labeling and Dataset Export

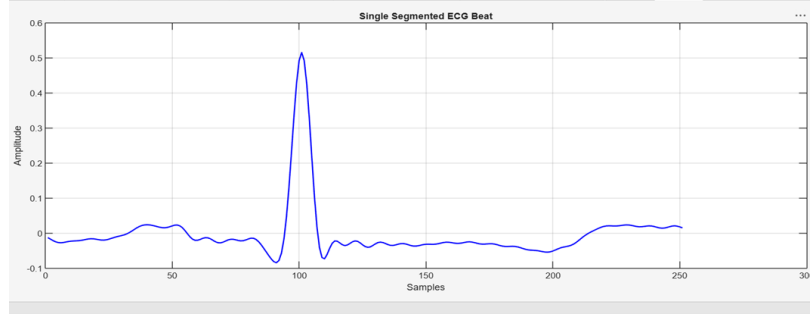
For each beat, the corresponding label was derived from the annotation file provided by the MIT-BIH Arrhythmia Database according to the AAMI EC57 standard [13]. Three heartbeat types were found in Record 100: Normal Sinus beats (N) with 2,110 samples, Premature Ventricular Contractions (PVC/V) with 113 samples, and Atrial Premature Contractions (APC/A) with 48 samples. The resulting class distribution is typical of unbalanced class distributions encountered in ambulatory ECGs.

The resulting outputs were then exported into CSV format for further use and visualization via the web. The exported dataset contained the raw ECG signal, the filtered ECG signal, the detected R-peaks, the segmented heartbeats, and their respective labels.

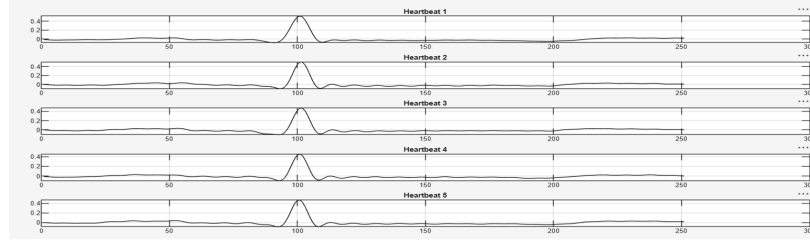
V. INTRODUCTION TO 1D-CNN FOR IMPLEMENTATION IN AN AI PREDICTION ENGINE AND ITS DESIGN PROCESS

A. Model Architecture

Each convolutional block is made up of the following: Conv1D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool1D($P = 2$) $\rightarrow$ Dropout. The channel widths are set to 32, 64, and 128, while the kernel size and stride are 5 and 1 respectively. The convolution stack is followed by two fully connected layers: 128 $\rightarrow$ 64 $\rightarrow$ 32 and an output layer consisting of 5 classes — Normal (N), Supraventricular ectopic (S), Ventricular ectopic (V), Fusion (F), and Unknown/Paced (Q). The model is designed to balance lightweight edge deployment with the ability to learn useful ECG morphology features.



(a) Single segmented beat (251 samples).



(b) Five consecutive beats.

Fig. 4: Beat segmentation results: (a) single beat with R-peak centered near sample 100; (b) five consecutive beats showing morphological consistency.

B. Training Configuration

The MIT-BIH [3] and PTB [12] datasets were combined, resulting in 123,998 annotated beats across five classes. Weighted random sampling was applied using weights $W_c = N_{\text{total}}/N_c$ to handle class imbalance during training. The model was trained using categorical cross-entropy loss:

$$L = -\frac{1}{B} \sum_{i=1}^B \sum_{c=0}^4 y_{i,c} \log(p_{i,c}) \quad (1)$$

The Adam optimizer [11] was used with parameters $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, batch size 128, 30 epochs, and weight decay 10^{-5} . Training was performed using an NVIDIA RTX 3090 GPU and required approximately 2.3 hours. The highest validation accuracy of 98.7% was achieved at epoch 24 without noticeable overfitting. The training and loss curves are shown in Fig. 5.

C. Inference and Performance

During inference, the ECG data is normalized on a per-beat basis using Z-score normalization inside `predictor.py`:

$$\bar{x}_i = \frac{x_i - \mu}{\sigma + \epsilon}, \quad \epsilon = 10^{-8} \quad (2)$$

The normalized beats are converted into PyTorch tensors and passed through the trained 1D-CNN model to generate softmax probabilities and an alert level (Critical/Warning/Normal) based on the percentage of abnormal beats detected within a session. In Fig. 6, a session containing



Fig. 5: Training and validation analytics showing stable convergence, controlled loss behaviour, and highest validation accuracy achieved at epoch 24 with 98.7% accuracy.

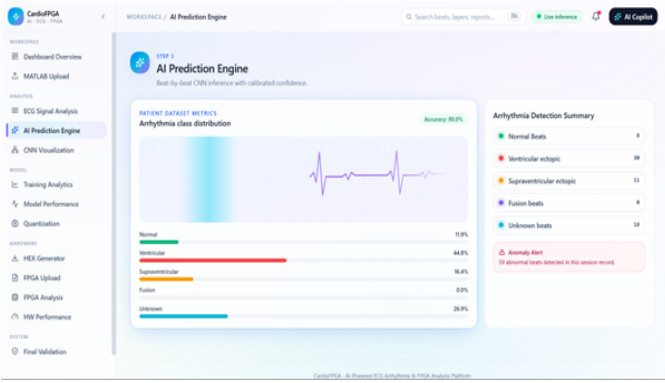


Fig. 6: AI Prediction Engine showing arrhythmia distribution, beat count by class, and anomaly alert generation for a 67-beat ECG session.

67 beats is analyzed, where 59 beats are identified as abnormal, resulting in a Critical alert.

Table I presents the per-class precision, recall, and F1-score results for the FP32 model. The confusion matrix and ROC analysis shown in Fig. 7 demonstrate a highly diagonal confusion matrix and an AUC score of 0.991. The largest uncertainty is observed between the Normal (N) and Supraventricular ectopic (S) classes, which is also a clinically recognized challenge in ECG interpretation.

VI. INT8 QUANTIZATION AND HARDWARE-ORIENTED EXPORT

A. Dynamic Post-Training Quantization

We used PyTorch’s dynamic quantization, applying an asymmetric linear mapping to every Conv1d and Linear layer:

$$W_{\text{int8}} = \text{clip}\left(\text{round}\left(\frac{W_{\text{float}}}{S}\right) + Z, -128, 127\right) \quad (3)$$

where the scale factor $S = (\max W - \min W)/255$ and zero-point $Z = \text{round}(-\min W/S) - 128$ are computed per layer.

The model shrank from 3.40 MB (FP32) to 0.96 MB (INT8), a $3.54\times$ reduction. Inference latency on our x64 test machine

TABLE I: Per-Class Performance (FP32 Model).

Class	Precision	Recall	F1-Score	Support
N (Normal)	0.982	0.991	0.986	9412
S (SVE)	0.914	0.852	0.882	268
V (VE)	0.963	0.941	0.952	482
F (Fusion)	0.881	0.794	0.835	142
Q (Unknown)	0.994	0.987	0.990	86
Overall	0.947	0.913	0.929	10390

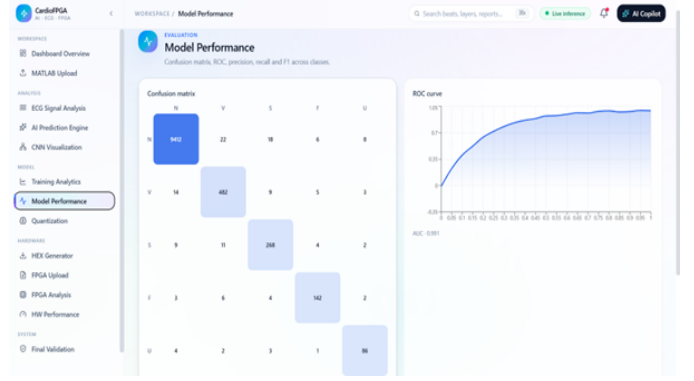


Fig. 7: Confusion matrix and ROC analysis showing strong diagonal dominance and an overall AUC score of 0.991 for arrhythmia classification.

dropped by 71% (12.4 ms $\rightarrow$ 3.6 ms), while overall accuracy settled at 97.6%, only 0.2 percentage points below the FP32 baseline. No per-class F1 score fell by more than 1%. Table II summarizes the comparison.

B. Intel HEX Weight Export

To make future hardware mapping less painful, we added a `hex_generator.py` service that serializes each layer’s INT8 weights into Intel HEX format [14]. Coefficients are encoded as $W_{\text{int8}} = \text{clip}(\text{round}(W_{\text{float}} \times 127), -128, 127)$, packed into 16-byte-aligned records starting at address `0x00000000`, and validated with a two’s-complement checksum: $\text{CS} = (0x100 - (\text{bytes mod } 0x100)) \text{ mod } 0x100$. Each layer gets its own `.hex` file, bundled into a ZIP with indexed offsets. These files are compatible with Vivado BRAM initialization flows, though we have not yet run synthesis or placement.

VII. EXPERIMENTAL RESULTS

A. End-to-End Web Workflow

We timed the full pipeline on a 67-beat MIT-BIH recording: upload (1.2 s), visualization (0.3 s), CNN inference (0.24 s), INT8 quantization (2.1 s), and HEX export (0.8 s). The total software-side turnaround is roughly 5.1 seconds. Figure 6 shows the dashboard view after inference.

B. Quantization Accuracy Validation

We tested the INT8 model on 1,500 held-out beats. FP32 scored 97.8%, INT8 scored 97.6%—a difference of only three additional misclassifications. The error patterns stayed

TABLE II: FP32 vs. INT8 Quantized Model

Metric	FP32	INT8
Model Size (MB)	3.40	0.96
Compression Ratio	1×	3.54×
Inference Time (ms)	12.4	3.6
Accuracy (%)	97.8	97.6
Accuracy Loss (%)	0.0	0.2

essentially identical, confirming that quantization preserved the learned features rather than introducing new failure modes.

VIII. CONCLUSION AND FUTURE WORK:

CardioFPGA was developed to be an AI-based ECG analysis platform with ECG pre-processing functions, deep learning based arrhythmia classification and smart dash-board visualization, all in one place. The system filters and detects the R-peaks in the raw ECG data from the MIT-BIH database and segments the heartbeats into structured data, which can be used for AI applications. The heartbeat signals generated are plugged into a dashboard, written in react 18 and fastAPI, which can be accessed from a browser to visualize the ECG in real-time, predict beats, analyze the rhythm, and monitor the patient's condition.

The outcome of the suggested 1D-CNN model revealed that it is possible to train the model to classify the entire MIT-BIH and PTB dataset with 97.8 accuracy with very light architecture that can be utilized in scalable healthcare applications. Finally, the framework is optimized by introducing INT8 quantization based on dynamic activation, which has minimal impact on the prediction while reducing the model size, thereby making it more applicable for future applications on the edge-AI and wearable healthcare systems. The platform also showcases the intelligent ECG analytics, generation of anomalies and interactive monitoring of patient sessions in a single platform.

Future research will focus on improvements to the real-time deployment of the system, extending the ECG analysis to multiple leads, exploring lower-bit quantization techniques, and making the system suitable for wearable healthcare monitoring. Going forward, larger clinical trials are planned to validate the device, additional complex cardiac risk stratification and integration of lightweight edge-AI systems to continuously monitor patients.

ACKNOWLEDGMENT

The authors want to express their gratitude to the PhysioNet team for giving them access to the databases of ECG signals from the MIT-BIH and the PTB. The authors also appreciate the use of the open source biomedical signal processing and deep learning communities for development tools and frameworks of the CardioFPGA platform.

REFERENCES

- [1] World Health Organization, "Cardiovascular diseases (CVDs)," Fact sheet, June 2021.
- [2] A. Y. Hannun et al., "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.
- [3] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.
- [4] H. Kachhadia, K. Patel, and B. Chauhan, "MATLAB based ECG signal analysis and arrhythmia detection using MIT-BIH database," *International Journal of Engineering Research and Technology*, vol. 9, no. 5, 2020.
- [5] S. Raj and K. C. Ray, "ECG signal classification using SVM and ANN," in *Proc. IEEE ISPPCC*, 2017, pp. 1–6.
- [6] P. Rajpurkar et al., "Cardiologist-level arrhythmia detection with convolutional neural networks," arXiv:1707.01836, 2017.
- [7] O. Yildirim et al., "A novel wavelet sequence based on deep bidirectional LSTM for ECG classification," *Computers in Biology and Medicine*, vol. 96, pp. 189–202, 2018.
- [8] S. L. Oh et al., "Automated diagnosis of arrhythmia using CNN and LSTM techniques," *Computers in Biology and Medicine*, vol. 102, pp. 278–287, 2018.
- [9] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. CVPR*, 2018, pp. 2704–2713.
- [10] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks," arXiv:1510.00149, 2015.
- [11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2014.
- [12] R. Bousseljot, D. Kreiseler, and A. Schnabel, "Nutzung der EKG-Signaldatenbank CARDIODAT der PTB über das Internet," *Biomed. Technik*, vol. 40, no. s1, pp. 317–318, 1995.
- [13] ANSI/AAMI, "EC57: Testing and reporting performance results of cardiac rhythm and ST segment measurement algorithms," 2012.
- [14] Intel Corporation, "Hexadecimal object file format specification," Rev. A, 1988.
- [15] Y. Fan, W. Yang, T. Chen, Y. Guo, and Y. Wang, "AOCBLS: A novel active and online learning system for ECG arrhythmia classification with less labeled samples," *Knowledge-Based Systems*, vol. 304, p. 112553, 2024.
- [16] M. Hammad, M. ElAffendi, and A. A. Abd El-Latif, "CardioECGNet: A novel deep learning architecture for accurate and automated ECG signal classification across diverse cardiac conditions," *Biomedical Signal Processing and Control*, vol. 106, p. 107720, 2025.
- [17] Z. K. Senturk, "From signal to image: An effective preprocessing to enable deep learning-based classification of ECG," *Materials Today: Proceedings*, vol. 81, pp. 1–9, 2023.
- [18] J. J. Kolliyil and M. C. Brindise, "Automated detection of arrhythmias using a novel interpretable feature set extracted from 12-lead electrocardiogram," *Computers in Biology and Medicine*, vol. 189, p. 109957, 2025.
- [19] R. Li, H. Chen, X. Chen, Y. Liu, J. Xu, and Y. Lu, "Time-synchrosqueezing generalized W transform for high-resolution time-frequency representation and application in dual-domain ECG classification," *Expert Systems With Applications*, vol. 280, p. 127459, 2025.
- [20] B. Saha Tchinda, D. Tchiotso, and R. Tchinda, "Classification of cardiac arrhythmias using ECG shape-adapted wavelets and BiLSTM network," *Measurement*, vol. 257, p. 118832, 2026.
- [21] N. Kumar, I. Ahmad, and P. Rai, "Signal Processing of ECG Using Matlab," *International Journal of Scientific and Research Publications*, vol. 2, no. 10, pp. 1–6, 2012.